



Universidade do Minho

Escola de Engenharia

Licenciatura em Engenharia informática

Mestrado Integrado em Engenharia Informática

Unidade Curricular de Scripting no Processamento de Linguagem Natural

Ano Lectivo de 2025/2026

Implementação prática de uma pipeline de information retrieval

Tomás Pinto Rodrigues (a104448),

25 Maio, 2026

SPLN

Índice

1. Introdução	1
2. Construção do Corpus	2
2.1. Recolha Automática de Dados	2
2.2. Extração de Links Dinâmicos	2
2.3. Limpeza e Processamento de HTML	3
2.4. Filtragem de Ruído	3
2.5. Armazenamento do Corpus	3
3. Processamento e Construção do Corpus	5
3.1. Leitura dos Documentos	5
3.2. Limpeza do Texto	6
3.3. Geração de Chunks	6
3.4. Resultados Obtidos	7
4. Módulo de Retrieval Lexical (TF-IDF)	8
4.1. Objetivo	8
4.2. Representação TF-IDF	8
4.3. Estratégia de Retrieval	8
4.4. Vantagens e Limitações	9
5. Módulo de Retrieval Semântico (S-BERT)	10
5.1. Objetivo	10
5.2. Geração de Embeddings	10
5.3. Estratégia de Retrieval	10
5.4. Vantagens e Limitações	11
6. Módulo de Question Answering Extrativo	12
6.1. Modelo utilizado	12
6.2. Funcionamento do sistema	12
6.3. Melhorias e heurísticas aplicadas	12
6.4. Seleção de evidência	13
7. Módulo de Question Answering Abstrativo	14
7.1. Modelo utilizado	14
7.2. Funcionamento do Sistema	14
7.3. Geração com FLAN-T5	14
7.4. Filtros e Controlo de Qualidade	15
8. Conclusão	16

1. Introdução

O presente trabalho foi desenvolvido no âmbito da unidade curricular de SPNL e tem como objetivo a construção de um sistema de Question Answering (QA) aplicado a um corpus textual sobre o Eurovision Song Contest.

O sistema desenvolvido segue uma arquitetura baseada em Retrieval-Augmented Generation (RAG), combinando técnicas de recuperação de informação com modelos de linguagem para responder a perguntas formuladas pelo utilizador.

O trabalho encontra-se dividido em três componentes principais:

- construção e processamento de um corpus textual;
- desenvolvimento de um módulo retriever para pesquisa de documentos relevantes;
- implementação de dois sistemas de QA: um extrativo e um abstrativo.

O corpus utilizado foi construído a partir de documentos relacionados com o Eurovision Song Contest contendo informação histórica sobre edições, participantes, vencedores e resultados.

O sistema final permite ao utilizador colocar perguntas em linguagem natural, sendo capaz de recuperar contexto relevante e gerar respostas automaticamente.

2. Construção do Corpus

2.1. Recolha Automática de Dados

A primeira etapa do projeto consistiu na construção de um corpus textual relacionado com o universo Eurovision.

Para esse efeito foi desenvolvido um *scraper* em Python capaz de recolher automaticamente informação textual a partir de páginas da Wikipédia relacionadas com:

- Eurovision Song Contest;
- Junior Eurovision Song Contest;
- concursos derivados (Eurovision Dance Contest, American Song Contest e Eurovision Asia).

O sistema utiliza a biblioteca *urllib* para descarregar o conteúdo HTML das páginas.

Foi também definido um User-Agent personalizado para garantir pedidos HTTP mais adequados e compatíveis com as políticas da Wikipédia.

2.2. Extração de Links Dinâmicos

O scraper foi desenvolvido para identificar automaticamente as páginas correspondentes a cada edição anual do concurso.

Para isso foram utilizadas expressões regulares capazes de detetar links válidos para páginas dos diferentes anos.

Exemplo:

- Eurovision Song Contest 1956–2026;
- Junior Eurovision Song Contest 2003–2026.

Esta abordagem permitiu automatizar totalmente a recolha de documentos sem necessidade de inserir manualmente cada URL.



```
00_extrai.py

regex_esc = re.compile(
    r'href="/wiki/Eurovision_Song_Contest_(195[6-9]|19[6-9]\d|20[0-1]\d|202[0-6])"'
)
```

Figura 1 - Regex para links

2.3. Limpeza e Processamento de HTML

Após o descarregamento das páginas, foi implementado um processo de limpeza textual para remover elementos irrelevantes do HTML original.

Entre os elementos removidos encontram-se:

- scripts JavaScript;
- estilos CSS;
- tabelas;
- infoboxes;
- referências bibliográficas;
- menus laterais;
- hiperligações;
- elementos de navegação da Wikipédia.

A limpeza foi realizada maioritariamente através de expressões regulares.

Posteriormente, o sistema extraiu apenas os elementos textuais relevantes, nomeadamente:

- parágrafos (<p>);
- títulos (<h2> e <h3>);
- listas ().

O texto extraído foi convertido para formato textual simples, removendo tags HTML e normalizando espaços e caracteres especiais.

2.4. Filtragem de Ruído

Durante o desenvolvimento verificou-se que as páginas da Wikipédia contêm uma grande quantidade de ruído textual não relevante para o sistema QA.

Foram então adicionados filtros específicos para remover:

- menus de navegação;
- links globais da Wikipédia;
- secções de referências;
- listas de idiomas;
- conteúdo relacionado com edição de páginas.

Esta etapa permitiu melhorar significativamente a qualidade final do corpus.

2.5. Armazenamento do Corpus

Após o processamento, cada documento foi guardado individualmente em formato **.txt**.

Os ficheiros foram organizados numa pasta denominada fontes, contendo um documento por edição do concurso.

Exemplos:

- Eurovision_Song_Contest_2017.txt
- Junior_Eurovision_Song_Contest_2008.txt

A utilização de ficheiros independentes facilitou as etapas posteriores de chunking, retrieval e question answering.

```
A processar: Junior Eurovision Song Contest 2024...
-> Sucesso: Junior_Eurovision_Song_Contest_2024.txt
A processar: Junior Eurovision Song Contest 2025...
-> Sucesso: Junior_Eurovision_Song_Contest_2025.txt
A processar: Junior Eurovision Song Contest 2026...
-> Sucesso: Junior_Eurovision_Song_Contest_2026.txt
```

```
--- INICIANDO FESTIVAIS ESPECIAIS E SPIN-OFFS ---
A processar: Eurovision Dance Contest - General Page...
-> Sucesso: Eurovision_Dance_Contest_General.txt
A processar: Eurovision Dance Contest 2007...
-> Sucesso: Eurovision_Dance_Contest_2007.txt
A processar: Eurovision Dance Contest 2008...
-> Sucesso: Eurovision_Dance_Contest_2008.txt
A processar: American Song Contest...
-> Sucesso: American_Song_Contest.txt
A processar: Eurovision Song Contest Asia...
-> Sucesso: Eurovision_Song_Contest_Asia.txt
```

```
[Dataset Concluído] Todos os ficheiros foram gerados na pasta 'fontes'!
```

Figura 2 - Extração do Corpus

3. Processamento e Construção do Corpus

Depois da recolha automática dos documentos da Wikipédia, foi desenvolvido um módulo responsável pelo processamento textual e construção do corpus final utilizado pelo sistema de retrieval e question answering.

O objetivo desta fase foi transformar os documentos brutos extraídos da Wikipédia num conjunto de chunks textuais mais pequenos e semanticamente coerentes, adequados para indexação semântica através de embeddings S-BERT.

3.1. Leitura dos Documentos

O sistema percorre automaticamente todos os ficheiros .txt presentes na pasta fontes/, ignorando diretórios e ficheiros inválidos.

Cada documento é carregado para memória e armazenado numa estrutura com:

- identificador do documento (doc_id)
- nome do ficheiro de origem (fonte)
- texto completo (texto)

Esta estrutura permite manter a rastreabilidade entre os chunks gerados e o documento original.

Exemplo da estrutura utilizada:



```
{
  "doc_id": 44,
  "fonte": "Eurovision_Song_Contest_1996.txt",
  "texto": "Eurovision Song Contest 1996 Location
The 1996 contest took place in Oslo, Norway, following
the country's victory at the 1995 contest with..."
},
```

Figura 3 - Exemplo de um documento no json

3.2. Limpeza do Texto

Antes da divisão em chunks, foi aplicada uma fase de limpeza textual com o objetivo de normalizar os documentos e reduzir ruído desnecessário.

As principais operações realizadas foram:

- remoção de quebras de linha (\n)
- remoção de tabulações (\t)
- normalização de múltiplos espaços consecutivos
- remoção de espaços em branco no início e fim do texto

Esta etapa melhora a consistência dos dados e evita problemas durante a tokenização e geração de embeddings.

3.3. Geração de Chunks

Após a limpeza, cada documento foi dividido em chunks textuais menores.

A divisão em chunks é necessária porque:

- modelos transformer possuem limites máximos de tokens
- embeddings funcionam melhor em blocos menores e semanticamente consistentes
- chunks menores aumentam a precisão do retrieval

Foi utilizada uma estratégia de sliding window com overlap de 30 palavras, tendo cada chunk 150 palavras. O overlap permite preservar contexto entre chunks consecutivos, reduzindo perda de informação nas fronteiras do texto.

Cada chunk é armazenado desta maneira:



```
{
  "chunk_id": 1528,
  "doc_id": 74,
  "fonte": "Eurovision_Song_Contest_2026.txt",
  "texto": "Semi-final allocation draw The draw to
determine the participating countries' semi-finals took
place on 12 January 2026 at 19:00 CET, at the Vienna City
Hall..."
},
```

Figura 4 - Exemplo de um Chunk no json

3.4. Resultados Obtidos

Após o processamento:

- foram carregados mais de 100 documentos (corpus_processado.json)
- foram gerados aproximadamente 1500 chunks textuais (chunks.json)
- cada chunk ficou associado ao respetivo documento original

O ficheiro chunks.json tornou-se a principal base de dados textual do sistema de QA.

4. Módulo de Retrieval Lexical (TF-IDF)

4.1. Objetivo

Foi desenvolvido um primeiro módulo de retrieval baseado em TF-IDF (Term Frequency – Inverse Document Frequency), com o objetivo de criar uma abordagem lexical inicial para pesquisa de informação no corpus.

Este módulo serve como baseline clássico de Information Retrieval, permitindo posteriormente comparar com abordagens semânticas mais avançadas (S-BERT)

4.2. Representação TF-IDF

Cada chunk do corpus foi representado através de um vetor TF-IDF utilizando a biblioteca scikit-learn.

Foi utilizado o TfidfVectorizer com os seguintes parâmetros:

- lowercase = True
- ngram_range = (1, 3) (unigramas, bigramas e trigramas)
- max_df = 0.95
- min_df = 2

Justificação:

A utilização de n-gramas até trigramas permite capturar expressões relevantes no domínio da Eurovisão, tais como:

- “song contest”
- “winner of”
- “eurovision song contest”

Cada chunk do corpus foi transformado num vetor numérico, formando uma matriz TF-IDF.

4.3. Estratégia de Retrieval

O processo de recuperação segue os seguintes passos:

1. Pré-filtragem heurística

Antes do cálculo de similaridade, são aplicadas regras para reduzir o espaço de pesquisa:

- filtragem por ano (ex: 2017, 2008)
- separação entre Eurovisão normal e Junior
- exclusão de documentos irrelevantes quando possível

2. Vetorização da query

A pergunta do utilizador é transformada num vetor TF-IDF.

3. Cálculo de similaridade

A relevância entre query e documentos é calculada através de cosine similarity.

4. Re-ranking com heurísticas

Foi aplicado um sistema de bonus lexical, que reforça chunks com termos importantes como: winner, won, win, eurovision, song contest, final

Este re-ranking melhora especialmente perguntas factuais.

4.4. Vantagens e Limitações

Vantagens

- Simplicidade e eficiência
- Interpretabilidade total
- Boa baseline para comparação

Limitações

- Não captura semântica
- Sensível a sinónimos
- Dependente de correspondência lexical exata

```
Insere uma pergunta (ou 'sair'): who win 2017 eurovision?

===== RESULTADOS =====

Resultado #1
Score: 1.0854
Fonte: Eurovision_Song_Contest_2017.txt

Texto:
questioned by the police at the venue police office." Other awards In addition to the main winner's trophy, the Marcel Bezençon Awards and the Barbara Dex Award were contested during the 2017 Eurovision Song Contest. The OGAE, "General Organisation of Eurovision Fans" voting poll also took place before the contest. Marcel Bezençon Awards The Marcel Bezençon Awards, organised since 2002 by Sweden's then-Head of Delegation and 1992 representative Christer Björkman, and 1984 winner Richard Herrey, honours songs in the contest's final. The awards are divided into three categories: Artistic Award, Composers Award, and Press Award. The winners were revealed shortly before the final on 13 May. OGAE

=====

Resultado #2
Score: 1.0110
Fonte: Eurovision_Song_Contest_2017.txt

Texto:
after 1956. Miroshnychenko has previously co-hosted the Junior Eurovision Song Contest in 2009 and 2013. Promotional emojis It was announced on 30 April that the creative teams from both the Eurovision network and Twitter had worked together to create three emoji that would accompany specific promotional hashtags for the duration of the contest. The heart emoji would appear alongside #ESC2017 and #Eurovision, while the winners' trophy emoji would be used for #12Points and #douzepoints. The final emoji is the logo for the contest, which would appear alongside #CelebrateDiversity, the slogan of the contest. Opening and interval acts The EBU released details regarding the opening and interval a
```

Figura 5 - Exemplo TD-IDF

5. Módulo de Retrieval Semântico (S-BERT)

5.1. Objetivo

Após a implementação do retriever lexical baseado em TF-IDF, foi desenvolvido um segundo módulo de retrieval baseado em representações semânticas densas, utilizando o modelo S-BERT (Sentence-BERT).

Este módulo tem como objetivo ultrapassar as limitações do TF-IDF, permitindo capturar significado semântico e não apenas correspondência lexical.

Foi utilizado o modelo “all-MiniLM-L6-v2” (Sentence Transformers) pois este modelo transforma cada chunk do corpus num vetor denso (embedding), representando o seu significado semântico.

5.2. Geração de Embeddings

Cada chunk do corpus foi convertido num embedding vetorial através do S-BERT. onde:

- cada texto é mapeado para um vetor denso de dimensão d
- textos semanticamente semelhantes ficam próximos no espaço vetorial

Os embeddings foram pré-calculados e armazenados em disco (sbert_embeddings.pkl) para evitar recomputação.

5.3. Estratégia de Retrival

O processo de retrieval segue os seguintes passos:

1. Pré-filtragem heurística

Antes do cálculo de similaridade semântica, são aplicadas regras de filtragem:

- Detecção de ano (ex: 2017, 2008)
- separação entre Eurovisão e Junior Eurovision
- restrição ao subconjunto mais relevante do corpus

2. Vetorização da query

A pergunta do utilizador é convertida num embedding: $q = f(\text{query})$

3. Cálculo de similaridade semântica

A relevância entre a query e cada chunk é calculada usando cosine similarity.

4. Re-ranking lexical híbrido

Apesar de ser um modelo semântico, foi adicionado um bonus lexical leve, que reforça chunks com palavras-chave importantes como: winner, won, win, eurovision, song contest, final.

Isto cria um modelo híbrido (semântico + lexical leve).

5.4. Vantagens e Limitações

Vantagens:

- Captura semântica - “who won Eurovision 2017” e “winner of Eurovision 2017” -> são entendidas como equivalentes
- Robustez a sinónimos - não depende de palavras exatas
- Melhor desempenho em perguntas naturais

Limitações:

- maior custo computacional do que TF-IDF
- dependência de modelo pré-treinado
- ainda pode falhar em casos muito ambíguos sem contexto suficiente

```
Pergunta (ou 'sair'): who win 2017 eurovision?

===== RESULTADOS S-BERT =====

Resultado #1
Score: 1.5281
Fonte: Eurovision_Song_Contest_2017.txt

Texto:
questioned by the police at the venue police office." Other awards In addition to the main winner's trophy, the Marcel Bezençon Awards and t
he Barbara Dex Award were contested during the 2017 Eurovision Song Contest. The OGAE, "General Organisation of Eurovision Fans" voting poll
also took place before the contest. Marcel Bezençon Awards The Marcel Bezençon Awards, organised since 2002 by Sweden's then-Head of Delega
tion and 1992 representative Christer Björkman, and 1984 winner Richard Herrey, honours songs in the contest's final. The awards are divided
into three categories: Artistic Award, Composers Award, and Press Award. The winners were revealed shortly before the final on 13 May. OGAE

-----

Resultado #2
Score: 1.4737
Fonte: Eurovision_Song_Contest_2017.txt

Texto:
after 1956. Miroshnychenko has previously co-hosted the Junior Eurovision Song Contest in 2009 and 2013. Promotional emojis It was announced
on 30 April that the creative teams from both the Eurovision network and Twitter had worked together to create three emoji that would accom
pany specific promotional hashtags for the duration of the contest. The heart emoji would appear alongside #ESC2017 and #Eurovision, while t
he winners' trophy emoji would be used for #12Points and #douzepoints. The final emoji is the logo for the contest, which would appear along
side #CelebrateDiversity, the slogan of the contest. Opening and interval acts The EBU released details regarding the opening and interval a
```

Figura 6 - Exemplo S-BERT

6. Módulo de Question Answering Extrativo

Foi implementado um módulo de Question Answering (QA) extrativo, responsável por localizar diretamente a resposta dentro dos textos recuperados pelo sistema de retrieval.

6.1. Modelo utilizado

O modelo base utilizado foi:

- **deepset/roberta-base-squad2**

Este modelo foi escolhido por já estar pré-treinado no dataset SQuAD, permitindo extrair respostas diretamente de contexto sem necessidade de treino inicial.

Posteriormente, foi criado um modelo fine-tuned próprio:

- **qa_finetuned (SQuAD fine-tuning)**

Este modelo foi treinado para melhorar o desempenho em perguntas relacionadas com o domínio do projeto (Eurovision).

6.2. Funcionamento do sistema

O processo de QA segue os seguintes passos:

1. Recebe uma pergunta do utilizador.
2. O módulo de retrieval (TF-IDF ou S-BERT) devolve um contexto relevante.
3. O modelo de QA analisa o contexto e calcula:
 - logits de início (start)
 - logits de fim (end)
4. São selecionados os melhores spans válidos (limite de tamanho e tokens válidos).
5. A resposta é decodificada a partir do texto original.

6.3. Melhorias e heurísticas aplicadas

Para melhorar a qualidade das respostas, foram adicionadas várias heurísticas:

- Limitação do tamanho máximo da resposta (evitar respostas longas ou erradas)
- Filtro de spans inválidos (ex: fora do contexto)

- Penalização de respostas com palavras irrelevantes (ex: *points*, *contest*, *score*)
- Regras específicas para perguntas do tipo “winner”
- Função de correção (*repair_answer*) para forçar extração correta do vencedor quando o modelo falha

6.4. Seleção de evidência

Antes da resposta final, o contexto é refinado com *select_evidence*, que:

- Divide o texto em frases
- Dá prioridade a frases com:
 - anos mencionados
 - palavras como “won”, “winner”
 - padrões de vitória (ex: “won the Eurovision Song Contest”)
- Remove ruído de Junior Eurovision quando não relevante

Assim, este modulo permite responder perguntas factuais com precisão

```
Pergunta (ou 'sair'): who win 2017 eurovision

--- CONTEXTO USADO ---
Portugal won with 758 points, winning both the jury vote and the televote.
-----

===== RESPOSTA =====
Resposta: Portugal
Score: 4.8869
=====
```

Figura 7 - Exemplo 1 QA Extrativo

```
Pergunta (ou 'sair'): winner 2008 eurovision junior

--- CONTEXTO USADO ---
Georgia won with 154 points, with Ukraine, Lithuania, Malta, and Macedonia completing the top five.
-----

===== RESPOSTA =====
Resposta: Georgia
Score: 5.2288
=====
```

Figura 8 - Exemplo 2 QA Extrativo

7. Módulo de Question Answering Abstrativo

Foi desenvolvido um segundo módulo de Question Answering (QA) baseado em geração de texto (*abstractive QA*), com o objetivo de complementar o sistema extrativo e melhorar a flexibilidade das respostas.

7.1. Modelo utilizado

O modelo generativo utilizado foi:

- **google/flan-t5-base**

Este modelo foi escolhido por ser leve, eficiente e adequado para tarefas de *instruction-based question answering*.

Para o cálculo de similaridade semântica foi novamente utilizado:

- **all-MiniLM-L6-v2 (S-BERT)**

7.2. Funcionamento do Sistema

O módulo segue uma abordagem híbrida:

1. A pergunta é normalizada (lowercase e correções simples de escrita).
2. O sistema identifica informação estruturada relevante (ex: ano da edição).
3. É feito retrieval semântico com S-BERT sobre os chunks.
4. É aplicado um sistema de scoring híbrido:
 - similaridade semântica (cosine similarity)
 - bonus lexical baseado em palavras-chave (winner, won, Eurovision, etc.)
5. São selecionados os top-K chunks mais relevantes.
6. O contexto final é refinado com `select_evidence`, que:
 - divide o texto em frases
 - prioriza frases com “won”, “winner”, “points”, etc.
 - reduz ruído irrelevante

7.3. Geração com FLAN-T5

Se não for possível obter resposta determinística, é usado o modelo generativo:

- O contexto e a pergunta são inseridos num prompt controlado
- O prompt força respostas curtas e factuais:
 - “Answer using ONLY the context”
 - “If not found, reply ‘Não sei’”

A geração é feita com:

- beam search (num_beams=4)
- sem sampling (do_sample=False)
- limite de tokens (max_new_tokens=20)

7.4. Filtros e Controlo de Qualidade

Foram aplicadas várias estratégias para evitar respostas incorretas:

- Remoção de contexto irrelevante (junior vs ESC)
- Filtro de anos na recuperação
- Penalização de termos genéricos
- Evitar respostas longas ou vagas
- Prioridade a frases com padrões de vitória

```
Pergunta: who win 2017 eurovision?

===== RESPOSTA =====
Portugal won with 758 points
=====

Pergunta: winner 2009 eurovision junior?

===== RESPOSTA =====
The Netherlands won with 121 points
=====
```

Figura 9 - Exemplo 1 QA Abstrativo

```
Pergunta: eurovision lisbon?

--- CONTEXTO USADO ---
Following Sobral's triumph, the European Broadcasting Union (EBU)'s Executive Supervisor for the Eurovision Song Contest, Jon Ola Sand, issued the hosting invitation to RTP during the winner's press conference. The following day, the director-general of RTP, Nuno Artur Silva, confirmed that the broadcaster would organise the contest in 2018 and mentioned the Lisbon Arena as a likely venue to host the contest.
-----

===== RESPOSTA =====
Lisbon Arena
=====
```

Figura 10 - Exemplo 2 QA Abstrativo

8. Conclusão

Ao longo deste trabalho foi desenvolvido um sistema completo de Question Answering aplicado ao domínio do Eurovision Song Contest, seguindo uma arquitetura Retrieval-Augmented Generation (RAG).

Inicialmente foi construído um corpus textual através de scraping automático da Wikipédia, seguido de processos de limpeza, normalização e divisão em chunks textuais. Posteriormente foram implementados dois sistemas de retrieval:

- um modelo lexical baseado em TF-IDF;
- um modelo semântico baseado em S-BERT.

Os resultados mostraram que o retrieval semântico apresentou melhor desempenho em perguntas formuladas em linguagem natural, conseguindo capturar relações semânticas que o TF-IDF não consegue representar.

Na fase de Question Answering foram desenvolvidas duas abordagens distintas:

- QA extrativo, baseado em deepset/roberta-base-squad2 com fine-tuning adicional sobre SQuAD;
- QA abstrativo, utilizando o modelo generativo google/flan-t5-base.

Além dos modelos, foram também implementadas várias heurísticas e filtros para melhorar a qualidade das respostas, incluindo:

- filtragem por ano;
- separação entre Eurovision e Junior Eurovision;
- seleção de evidência relevante;
- bonus lexical;
- correção automática de respostas incorretas.

De forma geral, o sistema desenvolvido demonstrou ser capaz de responder automaticamente a perguntas factuais sobre a Eurovisão com resultados bastante satisfatórios, evidenciando a importância da combinação entre retrieval semântico e modelos de linguagem para sistemas modernos de Question Answering.